Risk-Based Testing (RBT) Framework

FrontEnd | FrontEnd - Impact assessment - DemoQA Book Store UI

FrontEnd | Automation Priority: DemoQA Book Store UI Impact vs Effort

1. Purpose

Risk-Based Testing (RBT) is a testing strategy that prioritizes testing efforts based on **the likelihood of failure and the business impact of that failure**.

Key Principle:
Ensures that **testing effort is proportional to business risk**, allowing teams to deliver **higher quality software while optimizing testing time and resources**.

Instead of testing all features with the same intensity, RBT ensures that:

- Critical features receive **maximum test coverage**
- Lower-risk areas receive **appropriate but lighter testing**
- QA effort is aligned with **business risk and system complexity**

Goal:
Deliver higher quality software by focusing testing where failures would cause the **greatest damage to users or the business**.

Benefits:

- Improved defect detection in critical areas
- Efficient use of QA resources
- Faster release cycles
- Increased confidence in system stability
- Alignment of testing effort with business priorities

---

2. Definition of Risk

In the context of software testing:

Risk = Likelihood of Failure × Impact of Failure

Where:

Parameter	Description
Likelihood	Probability that a feature or component may fail
Impact	Severity of consequences if that failure occurs

Both parameters are scored on a **1–5 scale**.

---

3. Risk Scoring Model

Risk Score is calculated as:

Risk Score = Likelihood × Impact

Score Range	Risk Level
1 – 4	Low Risk

5 – 9	Medium Risk
10 – 16	High Risk
17 – 25	Critical Risk

This scoring allows teams to **prioritize testing effort proportionally to risk**.

---

4. Risk Matrix

The Risk Matrix visually maps **Likelihood vs Impact** to determine the overall risk classification.

Likelihood → Impact ↓	1 Very Low	2 Low	3 Medium	4 High	5 Very High
5 Critical Impact	🟡	🟠	🔴	🔴	🔴
4 High Impact	🟡	🟡	🟠	🔴	🔴
3 Medium Impact	🟢	🟡	🟡	🟠	🔴
2 Low Impact	🟢	🟢	🟡	🟡	🟠
1 Minimal Impact	🟢	🟢	🟢	🟡	🟡

Legend

🟢 Low Risk
🟡 Medium Risk
🟠 High Risk
🔴 Critical Risk

---

5. Likelihood Criteria

Likelihood represents the **probability that a defect may occur**.

Score	Likelihood Description
1	Stable component with minimal changes
2	Minor changes, historically stable
3	Moderate complexity or occasional defects
4	Frequent changes or complex integration
5	New feature or highly complex component

Factors influencing likelihood:

- Code complexity
- Recent code changes
- Integration dependencies
- Developer familiarity
- Historical defect rate

---

6. Impact Criteria

Impact represents the **severity of consequences if a failure occurs**.

Score	Impact Description
1	Minor inconvenience for users
2	Limited functionality affected
3	User workflow partially disrupted
4	Major feature failure affecting users
5	Business-critical failure (revenue, security, or system availability)

Factors influencing impact:

- Revenue impact
- Security implications

- User experience degradation
- Regulatory/compliance risk
- Data integrity

---

7. Example Risk Assessment

Below is an example risk evaluation for common application modules.

Feature / Component	Likelihood	Impact	Risk Score	Risk Level
User Authentication	3	5	15	High
Payment Processing	5	5	25	Critical
User Profile Settings	2	2	4	Low
External API Integration	4	4	16	High
Reporting Module	3	3	9	Medium

---

8. Test Strategy Based on Risk Level

Testing depth and coverage are adjusted according to the calculated risk.

Risk Level	Testing Approach
Critical Risk	Extensive functional, regression, security, and performance testing
High Risk	Thorough functional and integration testing
Medium Risk	Standard functional testing
Low Risk	Basic validation and smoke testing

This ensures testing resources are **focused on areas with the highest business risk**.

---

9. RBT Implementation Process

The following process is used to apply Risk-Based Testing during the test planning phase.

Step 1 — Identify System Components

List all major features, modules, or services.

Step 2 — Assess Likelihood

Evaluate the probability of defects using complexity, change frequency, and historical data.

Step 3 — Assess Impact

Determine the severity of failure on users, business operations, or compliance.

Step 4 — Calculate Risk Score

Risk Score = Likelihood × Impact.

Step 5 — Classify Risk Level

Place the feature within the risk matrix.

Step 6 — Allocate Testing Effort

Prioritize testing based on the calculated risk level.